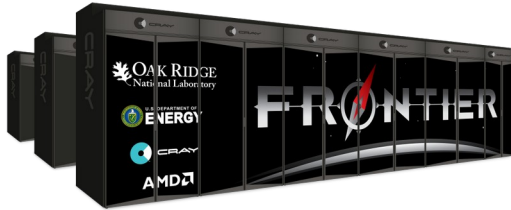


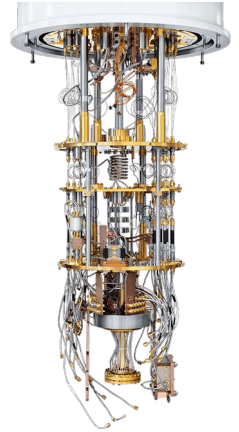
Workflows and System Architectures: QEC Perspective



Peter Groszkowski

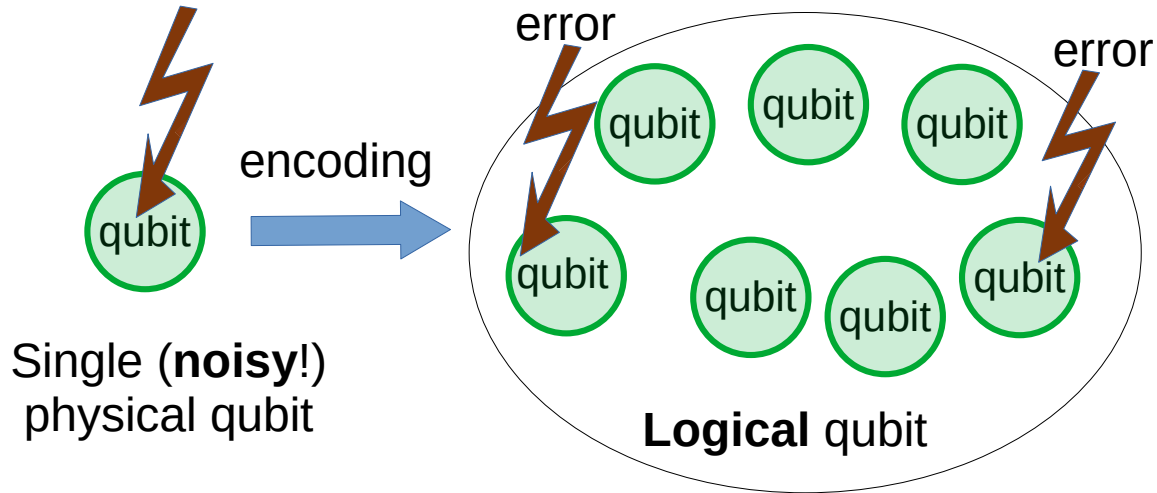
July 24, 2026

Workflows and Architectures Workstream
OpenQSE 2026



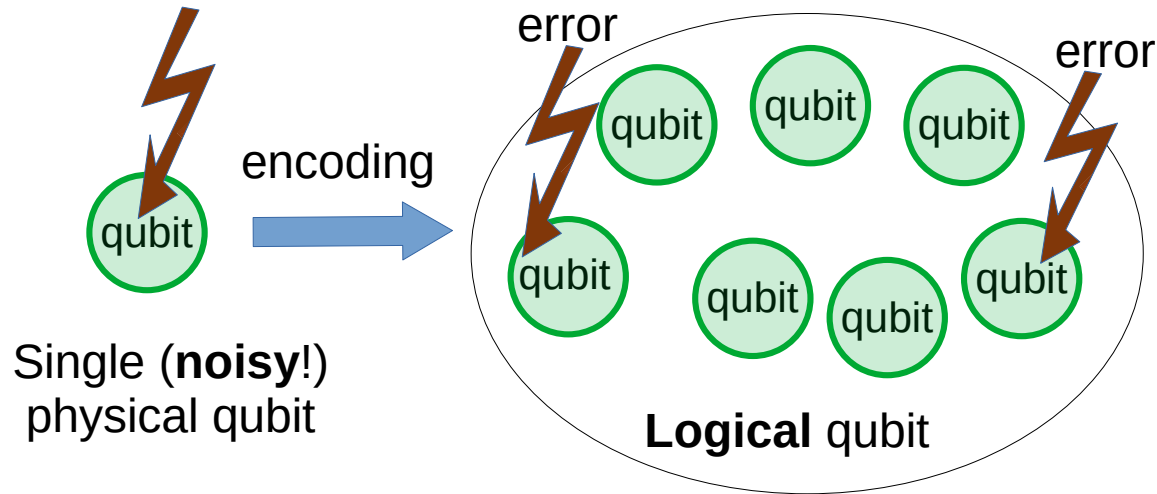
Quantum Error Correction 101

QEC provides a means to tolerate and overcome the effects of errors



Quantum Error Correction 101

QEC provides a means to tolerate and overcome the effects of errors



- QEC parametrized by

$[[n, k, d]]$

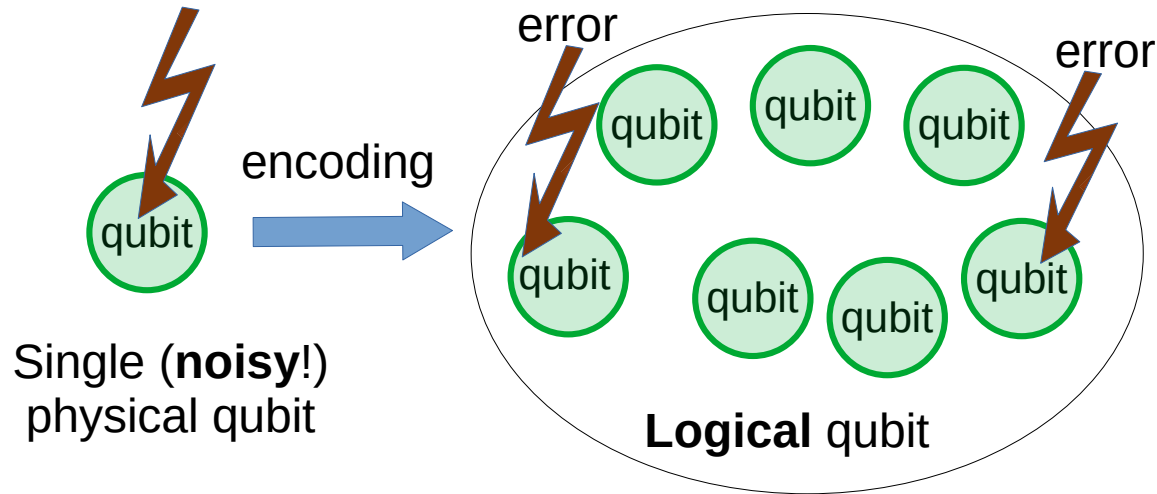
physical
qubit count

logical
qubit count

code
"distance"

Quantum Error Correction 101

QEC provides a means to tolerate and overcome the effects of errors



- QEC parametrized by

$[[n, k, d]]$

physical
qubit count

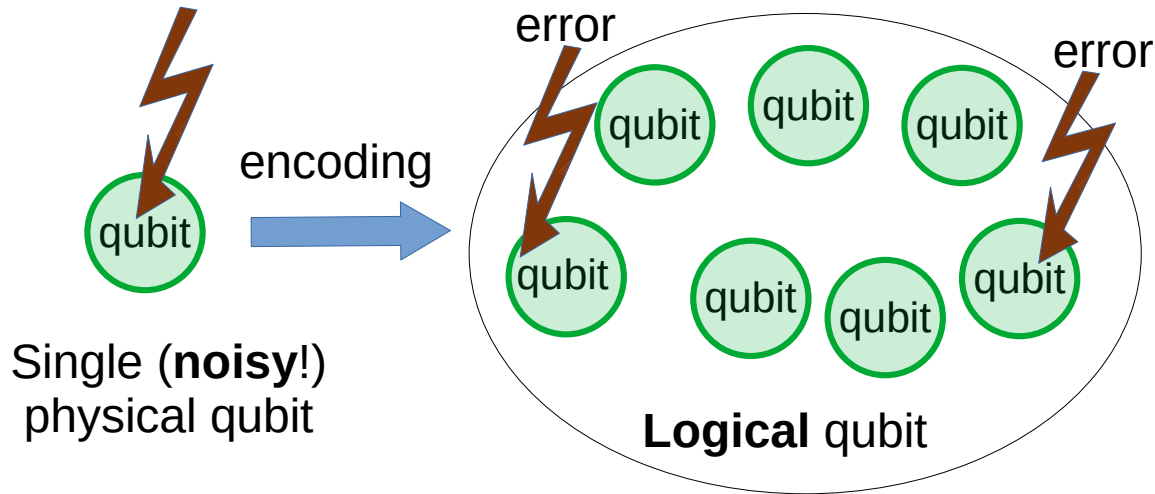
logical
qubit count

code
"distance"

- No such thing as "best" code for everyone!
 - hardware/resources constraints (noise type, gate-locality, etc)
 - ease of logical operations
 - error threshold is architecture/hardware dependent

Quantum Error Correction 101

QEC provides a means to tolerate and overcome the effects of errors



- QEC parametrized by

$[[n, k, d]]$

physical
qubit count

logical
qubit count

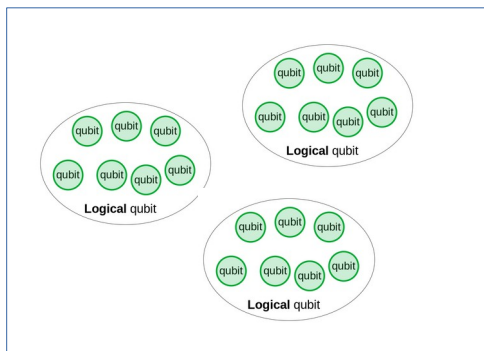
code
“distance”

- No such thing as “best” code for everyone!
 - hardware/resources constraints (noise type, gate-locality, etc)
 - ease of logical operations
 - error threshold is architecture/hardware dependent

**Different
QECs/architectures will
have different specific
requirements!**

QEC Implementation Workflow (Big Picture)

Loop
“indefinitely”:

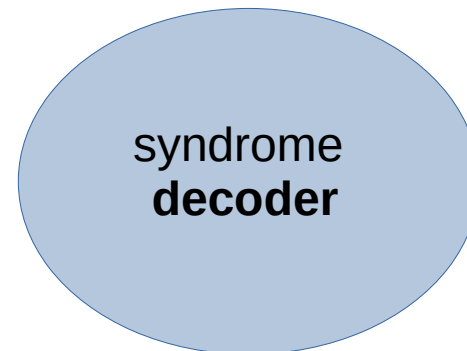


Quantum computer

measure syndrome

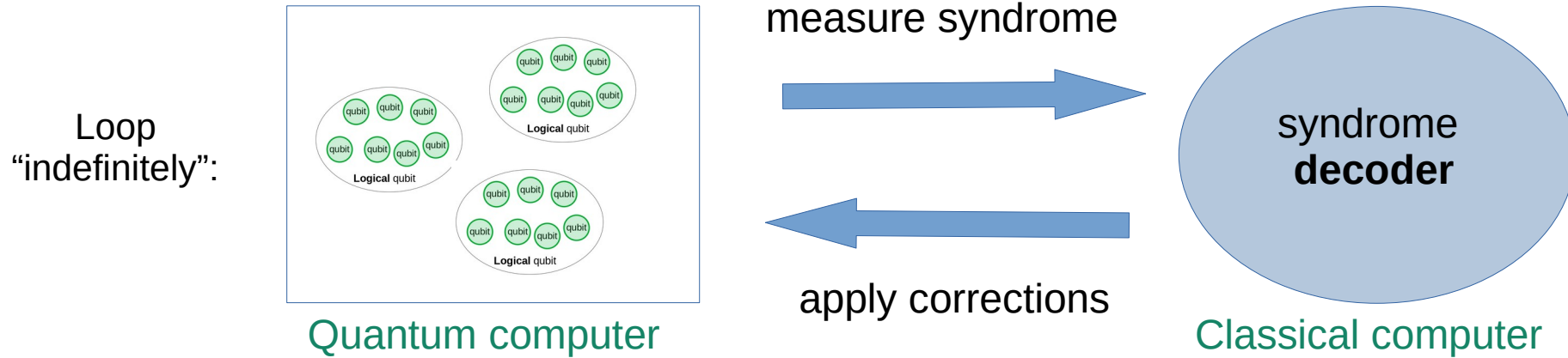


apply corrections

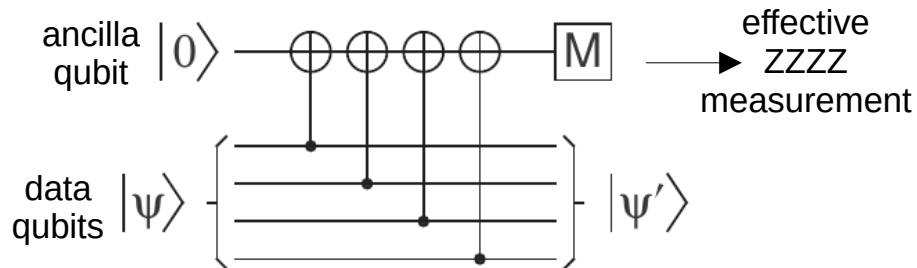


Classical computer

QEC Implementation Workflow (Big Picture)

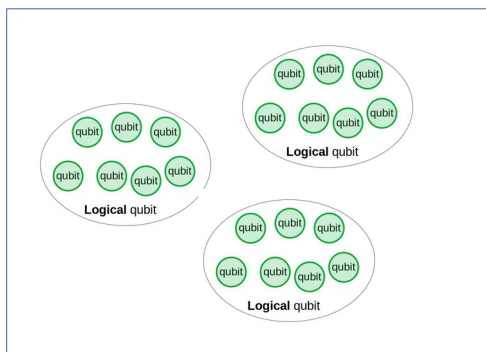


- Example of syndrome readout circuit



QEC Implementation Workflow (Big Picture)

Loop
“indefinitely”:

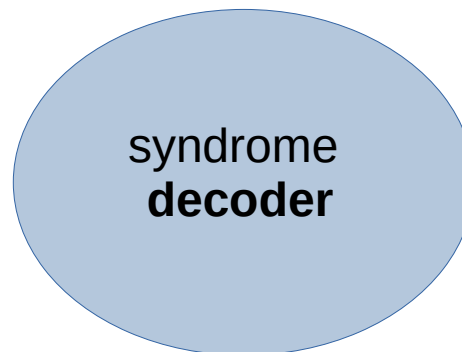


Quantum computer

measure syndrome

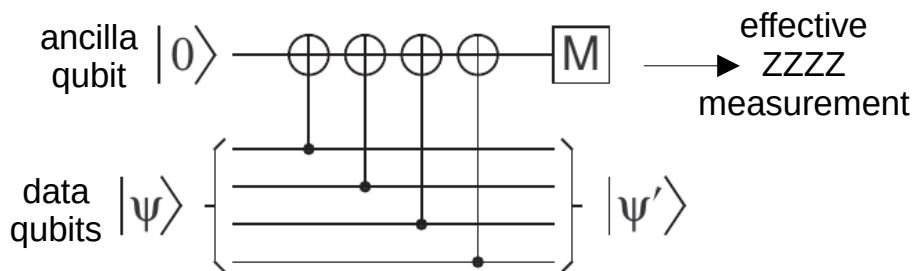


apply corrections



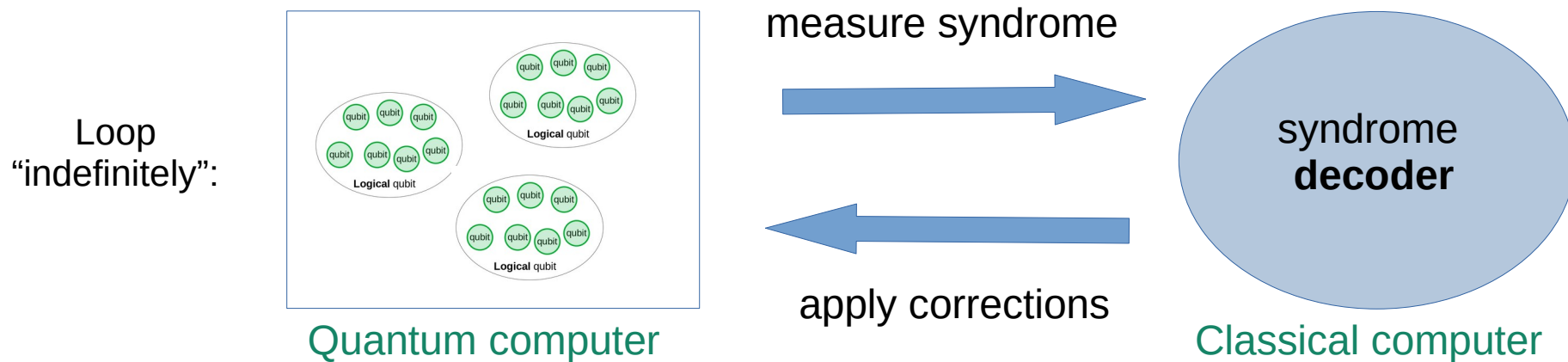
Classical computer

• Example of syndrome readout circuit

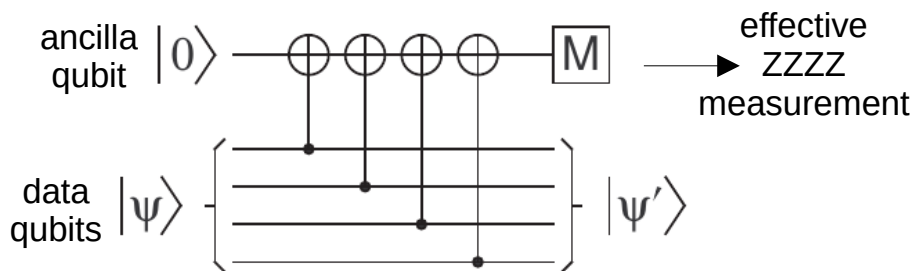


- Syndrome decoding tells us what corrections to apply
- In large(r) codes can be computationally “expensive”

QEC Implementation Workflow (Big Picture)



- Example of syndrome readout circuit



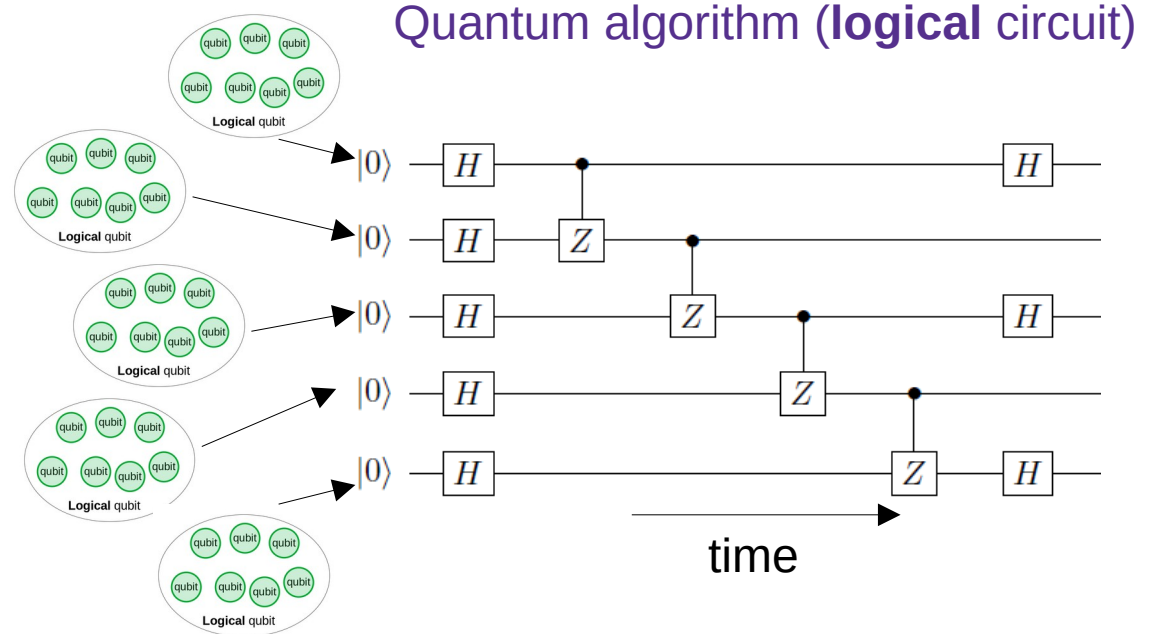
- Syndrome decoding tells us what corrections to apply
- In large(r) codes can be computationally “expensive”
- corrections can be typically deferred!

QEC: Memory vs Logic

- So far only discussed sustaining a qubit; **no logical gates**
 - Gives us logical **memory**
 - **Correction and decoding** can be deferred

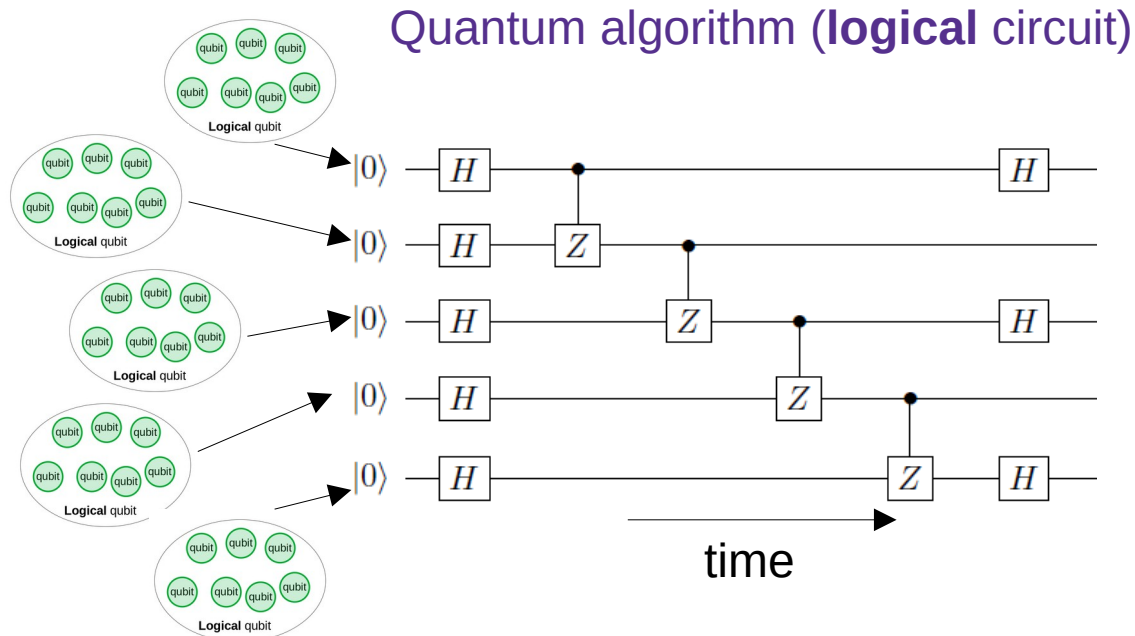
QEC: Memory vs Logic

- So far only discussed sustaining a qubit; **no logical gates**
 - Gives us logical **memory**
 - **Correction and decoding** can be deferred
- Long term goal to do computation
 - Need logical operations



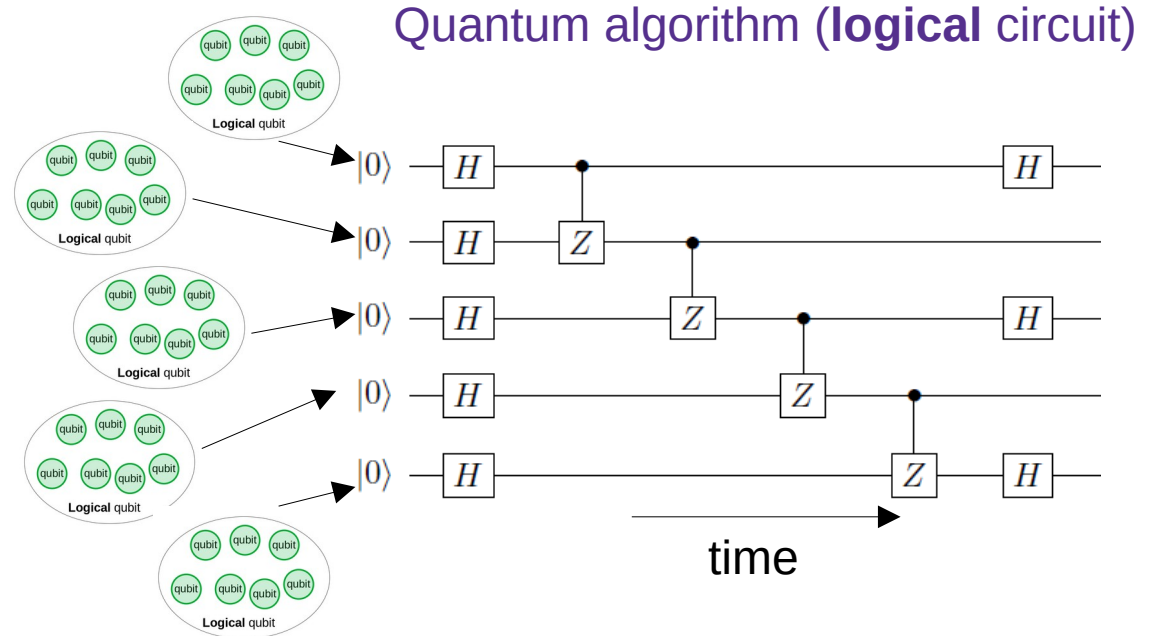
QEC: Memory vs Logic

- So far only discussed sustaining a qubit; **no logical gates**
 - Gives us logical **memory**
 - **Correction and decoding** can be deferred
- Long term goal to do computation
 - Need logical operations
- Logical operations generally “fit” between cycles



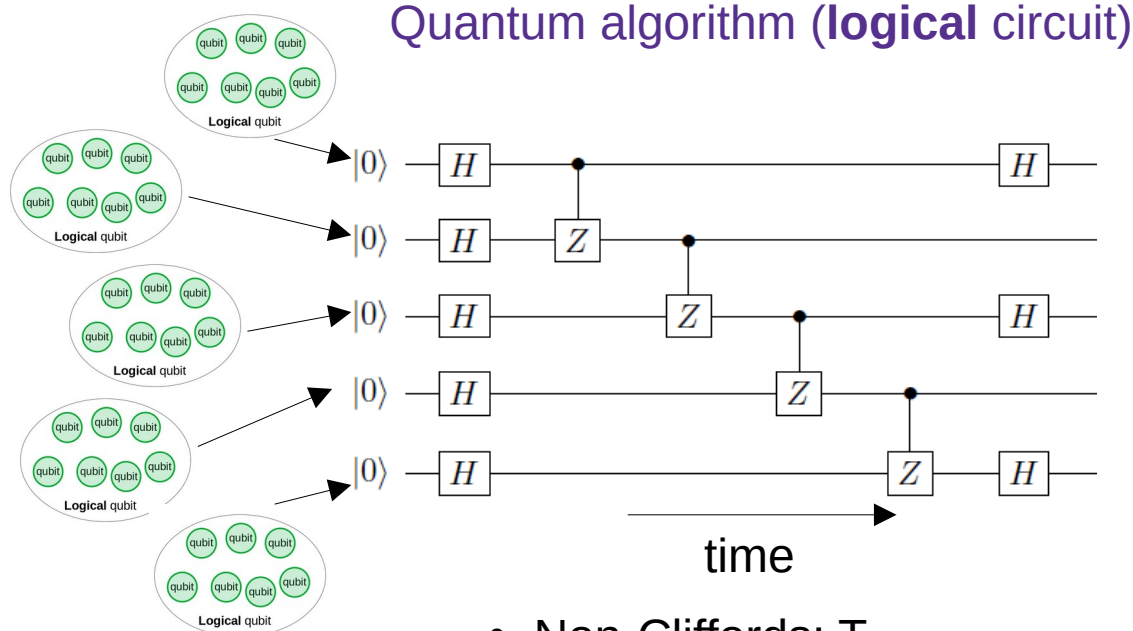
QEC: Memory vs Logic

- So far only discussed sustaining a qubit; **no logical gates**
 - Gives us logical **memory**
 - **Correction and decoding** can be deferred
- Long term goal to do computation
 - Need logical operations
- Logical operations generally “fit” between cycles
- Need to implement a **logical** universal gate set



QEC: Memory vs Logic

- So far only discussed sustaining a qubit; **no logical gates**
 - Gives us logical **memory**
 - **Correction and decoding can be deferred**
- Long term goal to do computation
 - Need logical operations
- Logical operations generally “fit” between cycles
- Need to implement a **logical** universal gate set



- Cliffords: H, S, CNOT
 - corrections and decoding can be deferred

- Non-Cliffords: T
 - **requires real-time decoding(!)**
 - **correction optional*****
 - greater latency constraints

Timing and Requirements

- What are the timing requirements?
 - Depends on architecture/code, but fast-enough so that **errors don't overwhelm QC**
(see later slides for example numbers)

Timing and Requirements

- What are the timing requirements?
 - Depends on architecture/code, but fast-enough so that **errors don't overwhelm QC**
(see later slides for example numbers)

Important time-scales and constraints considered when defining the system architecture:

- **qubit coherence**: need not “too many” errors per round
- **gate + readout + initialization times**: must fast-enough relative to coherence times
- **classical processing (decoding, pulse generation, etc)**: algorithm dependent, decoding usually harder with larger syndromes
- **data throughput**: lots of data to pass around

Timing and Requirements

- What are the timing requirements?
 - Depends on architecture/code, but fast-enough so that **errors don't overwhelm QC**
(see later slides for example numbers)

Important time-scales and constraints considered when defining the system architecture:

- **qubit coherence**: need not “too many” errors per round
- **gate + readout + initialization times**: must fast-enough relative to coherence times
- **classical processing (decoding, pulse generation, etc)**: algorithm dependent, decoding usually harder with larger syndromes
- **data throughput**: lots of data to pass around

In contrast to most application workflows (often no strict timing requirements) will likely need **more specialized hardware and software**

Riverlane's DeltaFlow

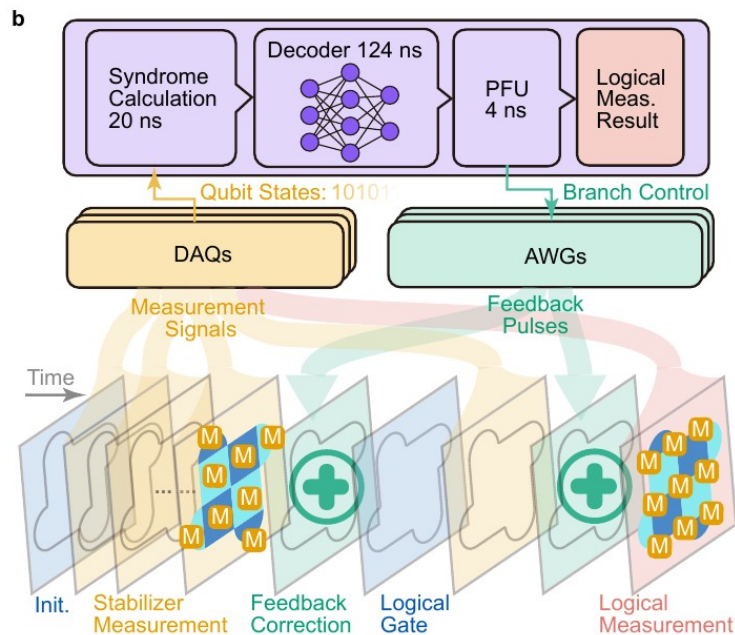
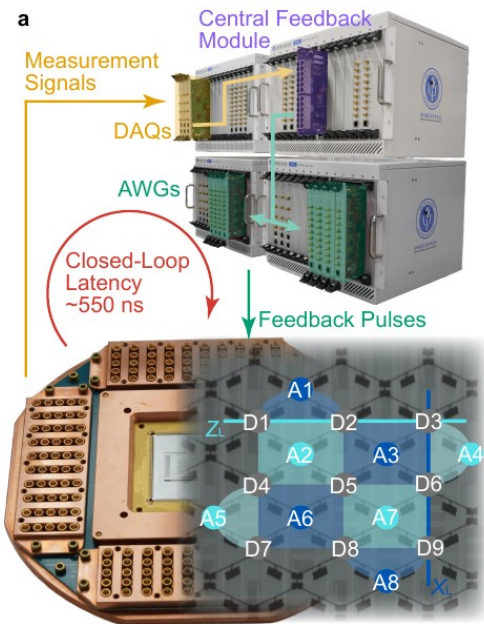


State of the Art: Recent Experiments

- Various recent **experimental** results in this space: IQM, Quantinuum, Riverlane, Rigetti, etc.

State of the Art: Recent Experiments

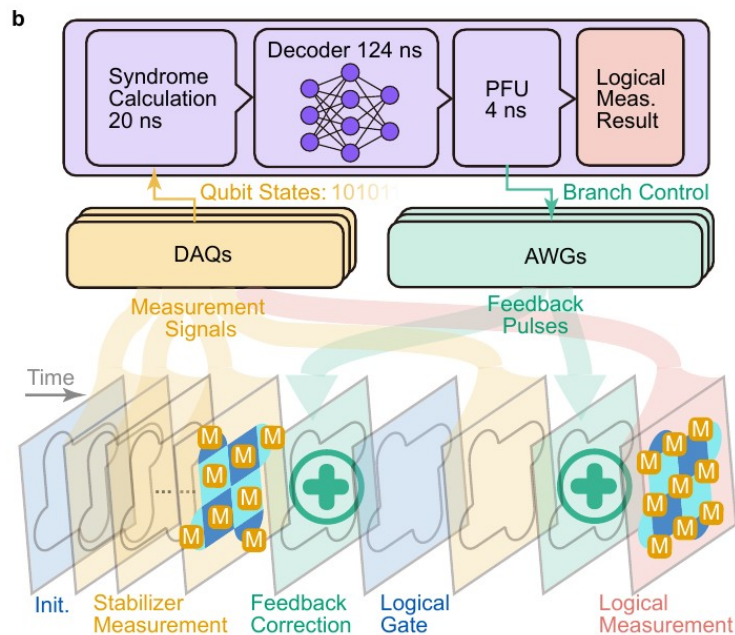
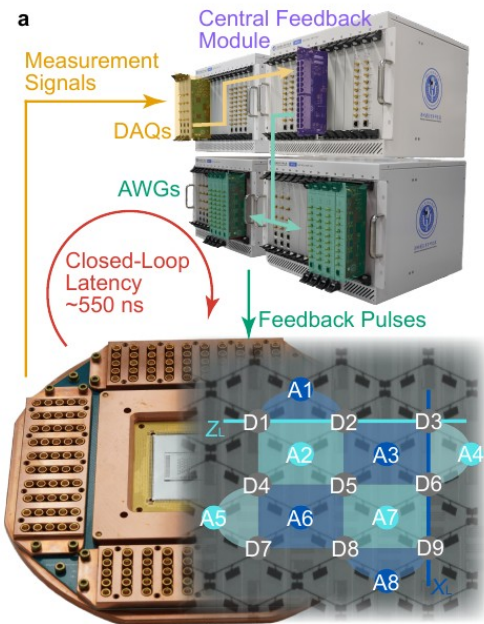
- Various recent **experimental** results in this space: IQM, Quantinuum, Riverlane, Rigetti, etc.



arxiv:2605.04892

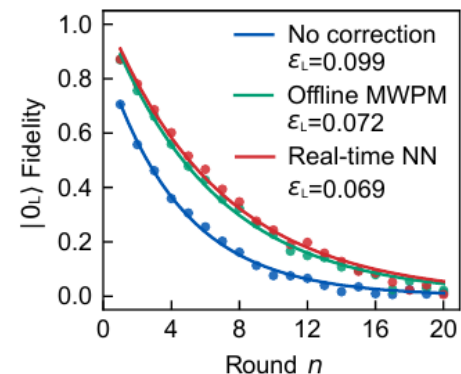
State of the Art: Recent Experiments

- Various recent **experimental** results in this space: IQM, Quantinuum, Riverlane, Rigetti, etc.



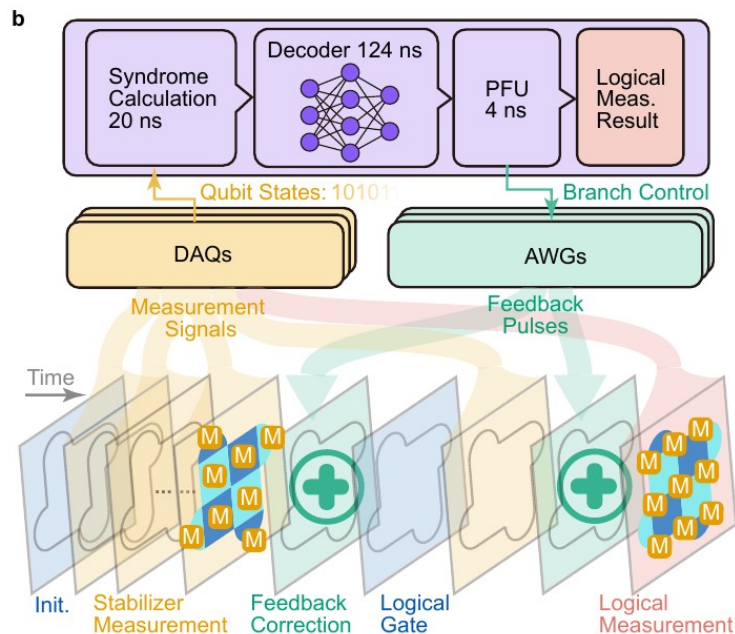
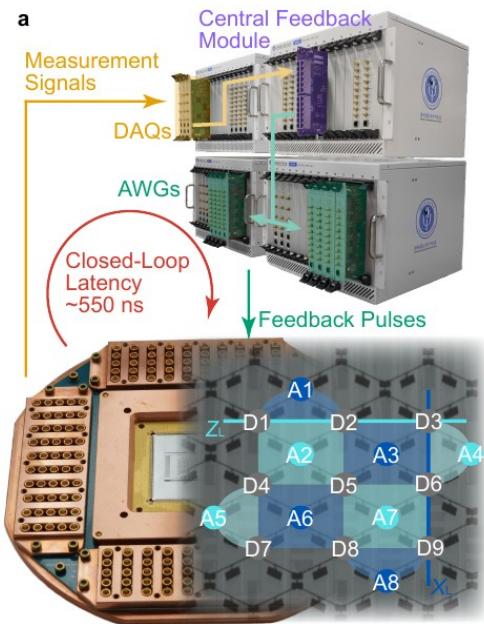
arxiv:2605.04892

Memory experiment



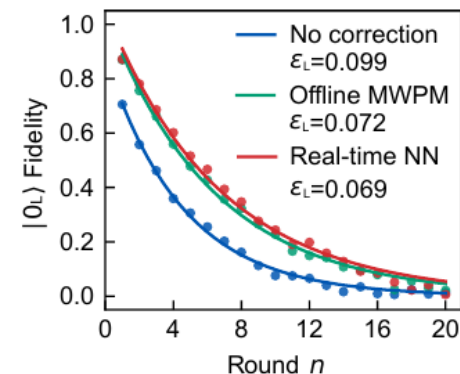
State of the Art: Recent Experiments

- Various recent **experimental** results in this space: IQM, Quantinuum, Riverlane, Rigetti, etc.



arxiv:2605.04892

Memory experiment



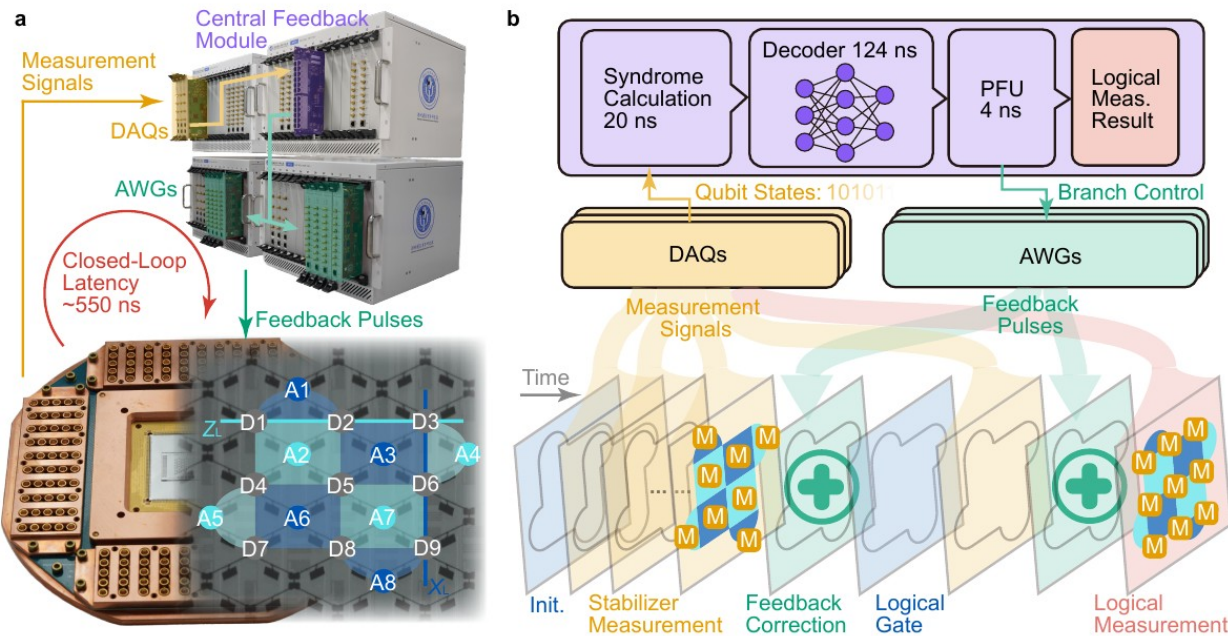
Classical-only cycle time: 550 ns

Category	Contribution	Latency (ns)
Readout sampling	DAQ sampling time after readout pulses	222
NN decoder	Syndrome calculation	20
	Neural network core latency	124
	Pauli frame update	4
	Subtotal (decoder)	148
Control electronics (excluding decoder)	ADC chip latency [65]	12
	IQ demodulation in DAQ FPGA	32
	Qubit-state classification logic	4
	Digital communication	36
	Backplane feedback logic latency	8
	Trigger propagation from backplane to AWG	16
	Waveform-generation logic in AWG FPGA	32
	DAC chip latency [66]	40
	Subtotal (excluding decoder)	180
Total closed-loop latency		550

Quantum part: $1.25 \mu\text{s}$

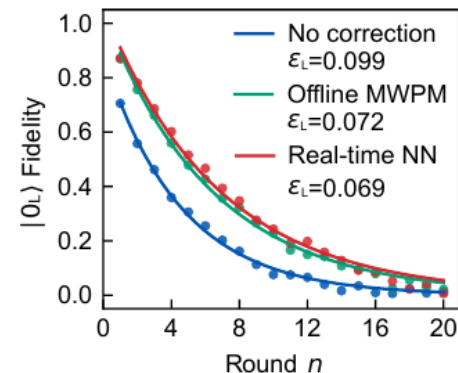
State of the Art: Recent Experiments

- Various recent **experimental** results in this space: IQM, Quantinuum, Riverlane, Rigetti, etc.



arxiv:2605.04892

Memory experiment



Classical-only cycle time: 550 ns

Category	Contribution	Latency (ns)
Readout sampling	DAQ sampling time after readout pulses	222
NN decoder	Syndrome calculation	20
	Neural network core latency	124
	Pauli frame update	4
	Subtotal (decoder)	148
Control electronics (excluding decoder)	ADC chip latency [65]	12
	IQ demodulation in DAQ FPGA	32
	Qubit-state classification logic	4
	Digital communication	36
	Backplane feedback logic latency	8
	Trigger propagation from backplane to AWG	16
	Waveform-generation logic in AWG FPGA	32
	DAC chip latency[66]	40
	Subtotal (excluding decoder)	180
Total closed-loop latency		550

- Shows applying QEC improves performance!
- Both memory and logical operations (limited non-Clifford)

Quantum part: $1.25 \mu\text{s}$

Parting Thoughts...

- QEC does not directly fall under the “application workflow” umbrella.
 - Applications will run “on top” of QEC protocols.
- ... but there is some overlap (i.e. need interleaved hybrid classical/quantum processing)
- QEC leads to most stringent timing requirements when compared to most applications
- Likely will be implemented directly by the vendors/hardware-makers with less “control” given to application devs (?)
- Are there direct overlaps in the near term (e.g., “hybrid” QEC+App. Workflow)?

The End